Answer Key: SQL Assignment (Sales)

For trainer use. MySQL 8.0.16 or later (CHECK constraints are enforced from this version).

Notes: IDs and error text in the key assume fresh tables and run in the order of the assignment. Error messages vary slightly by MySQL version and by schema name. Accept any answer that names the right error code and constraint. In MySQL Workbench, run `SET SQL_SAFE_UPDATES = 0;` before UPDATE or DELETE queries that have no key in the WHERE clause.

PART 1: Table Design & Constraints

Q1. Customer [1 mark]

```
CREATE TABLE Customer (
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,
    FullName   VARCHAR(50)  NOT NULL,
    Email      VARCHAR(100) NOT NULL UNIQUE,
    Phone      VARCHAR(15)  UNIQUE,
    Age        INT CHECK (Age >= 18),
    City       VARCHAR(50)  DEFAULT 'Kochi'
);
```

Marking: award the mark when all five rules are correct (PK + AUTO_INCREMENT, NOT NULL, UNIQUE on Email and Phone, CHECK, DEFAULT); deduct 0.25 for each missed rule.

Q2. Product [1 mark]

```
CREATE TABLE Product (
    ProductID    INT AUTO_INCREMENT PRIMARY KEY,
    ProductName  VARCHAR(100) NOT NULL,
    Brand        VARCHAR(50)  DEFAULT 'Dell',
    Price        DECIMAL(10,2) CHECK (Price > 0),
    StockQty     INT CHECK (StockQty >= 0),
    SerialNumber VARCHAR(30)  UNIQUE
);
```

Q3. SalesOrder and OrderItem [2 marks]

```
CREATE TABLE SalesOrder (
    OrderID    INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate  DATE NOT NULL DEFAULT (CURRENT_DATE),
    Status     VARCHAR(15) NOT NULL DEFAULT 'Pending'
               CHECK (Status IN ('Pending','Shipped','Delivered','Cancelled')),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
        ON DELETE CASCADE
);

CREATE TABLE OrderItem (
    OrderID   INT,
    ProductID INT,
    Quantity  INT CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) CHECK (UnitPrice > 0),
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (OrderID)   REFERENCES SalesOrder(OrderID) ON DELETE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)  ON DELETE RESTRICT
);
```

Marking: SalesOrder 1 mark (PK, FK with CASCADE, DEFAULT current date, Status CHECK + DEFAULT). OrderItem 1 mark (composite PK, two FKs with the correct actions, two CHECKs). Note that an expression default such as `DEFAULT (CURRENT_DATE)` needs the brackets in MySQL 8.

Q4. ALTER TABLE [1 mark]

```
ALTER TABLE OrderItem
    ADD COLUMN DiscountPercent DECIMAL(5,2) DEFAULT 0,
    ADD CONSTRAINT chk_discount CHECK (DiscountPercent BETWEEN 0 AND 50);

ALTER TABLE Customer
    ADD CONSTRAINT chk_phone CHECK (Phone REGEXP '^[0-9]{10}$');

ALTER TABLE Product DROP INDEX SerialNumber;   -- unnamed UNIQUE gets the column name
```

If the index name is unknown, run `SHOW INDEX FROM Product;` to find it. A NULL phone still passes the CHECK, because a CHECK fails only when the condition is FALSE, not NULL.

PART 2: INSERT & Constraint Violations

Q5. Customers and products [1 mark]

```
INSERT INTO Customer (FullName, Email, Phone, Age, City) VALUES
 ('Arun Kumar',  'arun@mail.com',   '9876543210', 28, 'Kochi'),
 ('Meera Nair',  'meera@mail.com',  '9876543211', 32, 'Thrissur'),
 ('Rahul Das',   'rahul@mail.com',  '9876543212', 25, 'Kozhikode'),
 ('Sneha Menon', 'sneha@mail.com',  '9876543213', 41, 'Kochi');

-- City column left out, so DEFAULT 'Kochi' applies
INSERT INTO Customer (FullName, Email, Phone, Age)
VALUES ('Vishnu P', 'vishnu@mail.com', '9876543214', 30);

INSERT INTO Product (ProductName, Brand, Price, StockQty, SerialNumber) VALUES
 ('Laptop',   'HP',       55000, 10, 'SN101'),
 ('Mouse',    'Logitech',  1200, 50, 'SN102'),
 ('Keyboard', 'Dell',      2500, 30, 'SN103'),
 ('Monitor',  'Samsung',  12000, 15, 'SN104');

-- Brand left out, so DEFAULT 'Dell' applies
INSERT INTO Product (ProductName, Price, StockQty, SerialNumber)
VALUES ('Webcam', 3000, 20, 'SN105');

SELECT * FROM Customer;
SELECT * FROM Product;
```

Key point: writing NULL explicitly does **not** trigger the DEFAULT. Leave the column out of the column list (or use the keyword DEFAULT).

Q6. Orders and order items [1 mark]

```
INSERT INTO SalesOrder (CustomerID, OrderDate, Status) VALUES
 (1, '2026-10-01', 'Delivered'),   -- order 1
 (1, '2026-10-15', 'Shipped'),     -- order 2
 (2, '2026-10-05', 'Delivered'),   -- order 3
 (3, '2026-10-20', 'Pending'),     -- order 4
 (4, '2026-10-22', 'Cancelled'),   -- order 5
 (2, '2026-11-01', 'Pending');     -- order 6

INSERT INTO OrderItem (OrderID, ProductID, Quantity, UnitPrice) VALUES
 (1, 1, 1, 55000), (1, 2, 2, 1200),
 (2, 3, 1, 2500),
 (3, 2, 3, 1200),  (3, 3, 1, 2500),
 (4, 4, 2, 12000),
 (5, 1, 1, 55000), (5, 5, 1, 3000),
 (6, 2, 1, 1200),  (6, 4, 1, 12000);
```

Q7. Constraint violations [2 marks: about 0.4 per case for the error and the constraint named]

```
-- (a) duplicate email
INSERT INTO Customer (FullName, Email, Phone, Age)
VALUES ('Test', 'arun@mail.com', '9000000001', 22);
-- ERROR 1062 (23000): Duplicate entry 'arun@mail.com' for key 'customer.Email'
-- Cause: UNIQUE constraint on Email

-- (b) Price = 0 and negative stock
INSERT INTO Product (ProductName, Price, StockQty, SerialNumber)
VALUES ('Cable', 0, 5, 'SN200');
-- ERROR 3819 (HY000): Check constraint 'product_chk_1' is violated.
-- Cause: CHECK (Price > 0)
INSERT INTO Product (ProductName, Price, StockQty, SerialNumber)
VALUES ('Cable', 100, -5, 'SN200');
-- ERROR 3819: Check constraint 'product_chk_2' is violated.
-- Cause: CHECK (StockQty >= 0)

-- (c) duplicate composite key
INSERT INTO OrderItem (OrderID, ProductID, Quantity, UnitPrice)
VALUES (1, 1, 1, 55000);
-- ERROR 1062 (23000): Duplicate entry '1-1' for key 'orderitem.PRIMARY'
-- Cause: composite PRIMARY KEY (OrderID, ProductID)

-- (d) customer that does not exist
INSERT INTO SalesOrder (CustomerID, OrderDate) VALUES (99, '2026-11-10');
-- ERROR 1452 (23000): Cannot add or update a child row: a foreign key
--   constraint fails (... FOREIGN KEY (CustomerID) REFERENCES customer ...)
-- Cause: FOREIGN KEY on SalesOrder.CustomerID

-- (e) invalid status
INSERT INTO SalesOrder (CustomerID, OrderDate, Status)
VALUES (1, '2026-11-10', 'Returned');
-- ERROR 3819: Check constraint 'salesorder_chk_1' is violated.
-- Cause: CHECK (Status IN (...))
```

CHECK names such as `product_chk_1` are auto-generated and numbered in the order the checks appear, so students may see different numbers. Servers older than MySQL 8.0.16 accept CHECK syntax but ignore it, so (b) and (e) would wrongly succeed there.

PART 3: UPDATE & DELETE

Q8. UPDATE queries [1.5 marks]

```
UPDATE Product SET Price = Price * 1.10 WHERE Brand = 'Dell';

UPDATE SalesOrder SET Status = 'Shipped' WHERE OrderID = 4;

-- No NULL city exists yet (the DEFAULT filled it), so insert one explicitly:
INSERT INTO Customer (FullName, Email, Phone, Age, City)
VALUES ('Anil Raj', 'anil@mail.com', '9876543215', 35, NULL);
UPDATE Customer SET City = 'Trivandrum' WHERE City IS NULL;
```

Common mistakes: writing `City = NULL` (never true; use IS NULL), and forgetting the WHERE clause so every row changes.

Q9. DELETE queries [1 mark]

```
DELETE FROM OrderItem WHERE OrderID = 6 AND ProductID = 4;

DELETE FROM SalesOrder WHERE Status = 'Cancelled';
SELECT * FROM OrderItem WHERE OrderID = 5;   -- empty set
```

Order 5 was cancelled, so its two items (products 1 and 5) disappear automatically through ON DELETE CASCADE on OrderItem.OrderID.

Q10. Deleting a customer with orders [1.5 marks]

```
SELECT * FROM SalesOrder WHERE CustomerID = 1;   -- orders 1 and 2
SELECT * FROM OrderItem  WHERE OrderID IN (1,2); -- 3 rows

DELETE FROM Customer WHERE CustomerID = 1;

SELECT * FROM SalesOrder WHERE CustomerID = 1;   -- empty
SELECT * FROM OrderItem  WHERE OrderID IN (1,2); -- empty
```

The delete cascades twice: Customer to SalesOrder (orders 1 and 2 removed), then SalesOrder to OrderItem (their 3 items removed). Marking: before/after output 0.5 mark, explanation of the two-level cascade 1 mark.

Q11. Deleting a sold product [1 mark]

```
DELETE FROM Product WHERE ProductID = 2;
-- ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key
--   constraint fails (... FOREIGN KEY (ProductID) REFERENCES product ...)

-- Steps to remove it:
DELETE FROM OrderItem WHERE ProductID = 2;   -- 1. remove the child rows
DELETE FROM Product   WHERE ProductID = 2;   -- 2. then the parent row
```

RESTRICT blocks deleting a parent row while child rows exist, protecting the sales history. In a real business, the better design is to mark the product as discontinued instead of deleting it.

PART 4: DELETE vs TRUNCATE vs DROP

Q12. TRUNCATE and DROP on a referenced table [1 mark]

```
TRUNCATE TABLE Customer;
-- ERROR 1701 (42000): Cannot truncate a table referenced in a foreign key
--   constraint (`shop`.`salesorder`, CONSTRAINT `salesorder_ibfk_1`)

DROP TABLE Customer;
-- ERROR 3730 (HY000): Cannot drop table 'customer' referenced by a foreign
--   key constraint 'salesorder_ibfk_1' on table 'salesorder'.
```

Why: both commands would leave SalesOrder rows pointing to customers that no longer exist (orphan rows), breaking referential integrity. TRUNCATE and DROP do not delete row by row, so they cannot apply the CASCADE rule. The safe fix is to remove the child table first. `SET FOREIGN_KEY_CHECKS = 0;` bypasses the check but is dangerous.

Q13. AUTO_INCREMENT behaviour [2 marks]

```
CREATE TABLE Product_Backup LIKE Product;
INSERT INTO Product_Backup SELECT * FROM Product;   -- ids 1,3,4,5 after Q11

-- Part 1: DELETE without WHERE
DELETE FROM Product_Backup;
INSERT INTO Product_Backup (ProductName, Price, StockQty, SerialNumber)
VALUES ('Test', 100, 1, 'T1');
SELECT ProductID FROM Product_Backup;      -- 6  (counter CONTINUES)

-- Part 2: TRUNCATE
INSERT INTO Product_Backup SELECT * FROM Product;   -- refill (ids 1,3,4,5)
TRUNCATE TABLE Product_Backup;
INSERT INTO Product_Backup (ProductName, Price, StockQty, SerialNumber)
VALUES ('Test', 100, 1, 'T1');
SELECT ProductID FROM Product_Backup;      -- 1  (counter RESETS)
```

Observation: DELETE removes rows but keeps the AUTO_INCREMENT counter, so the next ID continues from the highest value used. TRUNCATE drops and recreates the table's data, so the counter restarts at 1. Note that `CREATE TABLE ... AS SELECT` would copy only the data and none of the keys or constraints, which is why LIKE is used here.

Q14. Order of removal [1 mark]

```
DROP TABLE OrderItem;       -- child of both SalesOrder and Product
DROP TABLE SalesOrder;      -- child of Customer
DROP TABLE Customer;
DROP TABLE Product;
DROP TABLE Product_Backup;  -- no foreign keys, any position
```

Children must be dropped before their parents. OrderItem references two tables, so it goes first. Any order that drops a parent before its child fails with error 3730.

Q15. Scenario: accidental DELETE and TRUNCATE [1.5 marks]

Recoverable: the DELETE. DELETE is DML and is logged row by row, so ROLLBACK inside the open transaction restores the rows. TRUNCATE is DDL, causes an implicit commit in MySQL, and cannot be rolled back (recovery would need a backup).

Faster: TRUNCATE, because it deallocates the table's data in one operation instead of removing rows one at a time.

For only cancelled orders: `DELETE FROM SalesOrder WHERE Status = 'Cancelled';`, because only DELETE supports a WHERE clause.

Extra danger: `DELETE FROM SalesOrder;` also removes every OrderItem row through ON DELETE CASCADE, so the entire sales history is wiped, not just the orders. (In practice, TRUNCATE on SalesOrder would fail with error 1701 here because OrderItem references it, unless foreign key checks were disabled.)

PART 5: Keys & Theory

Q16. Keys of the Customer table [1.5 marks]

Super keys (any set containing a candidate key; any 5 correct ones): {CustomerID}, {Email}, {Phone}, {CustomerID, FullName}, {Email, Age}, {Phone, City}, {CustomerID, Email}, {Email, Phone}, {CustomerID, Email, Phone}, {CustomerID, FullName, Age, City}.

Candidate keys (minimal super keys): {CustomerID}, {Email}, {Phone}.

Primary key: CustomerID. It is short, numeric, auto-generated, never NULL and never changes. Email and Phone can change over time, and Phone is nullable, so they are better kept as alternate keys with UNIQUE.

Non-prime attributes (not part of any candidate key): FullName, Age, City.

Q17. Composite key in OrderItem [1.5 marks]

OrderID alone fails because one order can contain several products: order 1 has two rows (1,1) and (1,2), so OrderID repeats. ProductID alone fails because one product is sold in many orders: product 2 appears in orders 1, 3 and 6, so ProductID repeats. Only the pair (OrderID, ProductID) is unique, and it also stops the same product being added twice to one order, so a composite key is the right choice.

Q18. Yes or No [1.5 marks]

(a) Yes. Customer has three candidate keys: CustomerID, Email and Phone.

(b) No. A table has only one primary key, although that single key can be composite, as in OrderItem(OrderID, ProductID).

(c) Yes. OrderItem has two foreign keys: OrderID to SalesOrder and ProductID to Product.

(d) Yes. A foreign key may reference any UNIQUE key of the parent. For example, a table could reference Product(SerialNumber). Referencing the primary key is the recommended practice.

Q19. Foreign key vs primary key [1 mark]

Foreign key: can contain NULL (if the column allows it, meaning no parent) and can contain duplicates. For example, many orders can share one CustomerID. Non-NULL values must exist in the parent table.

Primary key: can never contain NULL and can never contain duplicates, since it identifies each row.

PRIMARY KEY vs UNIQUE: a table can have only one primary key but many UNIQUE constraints. A primary key rejects NULL, while a UNIQUE column accepts NULL (MySQL allows multiple NULLs). In InnoDB the primary key is also the clustered index that stores the rows.

Total: 25 marks. Award partial marks when the logic is right but the syntax has a small slip.